Lập trình mạng · Chương 2

TCP: kết nối giữa client và server

TS. Đặng Ngọc Hùng

Chương 2: Lập trình socket TCP và đa luồng

1 TCP

- Kết nối giữa client và server
- Timeout, half-close và lỗi kết nối
- Bài toán ứng dụng: dịch vụ tra cứu qua TCP

2 Đa luồng

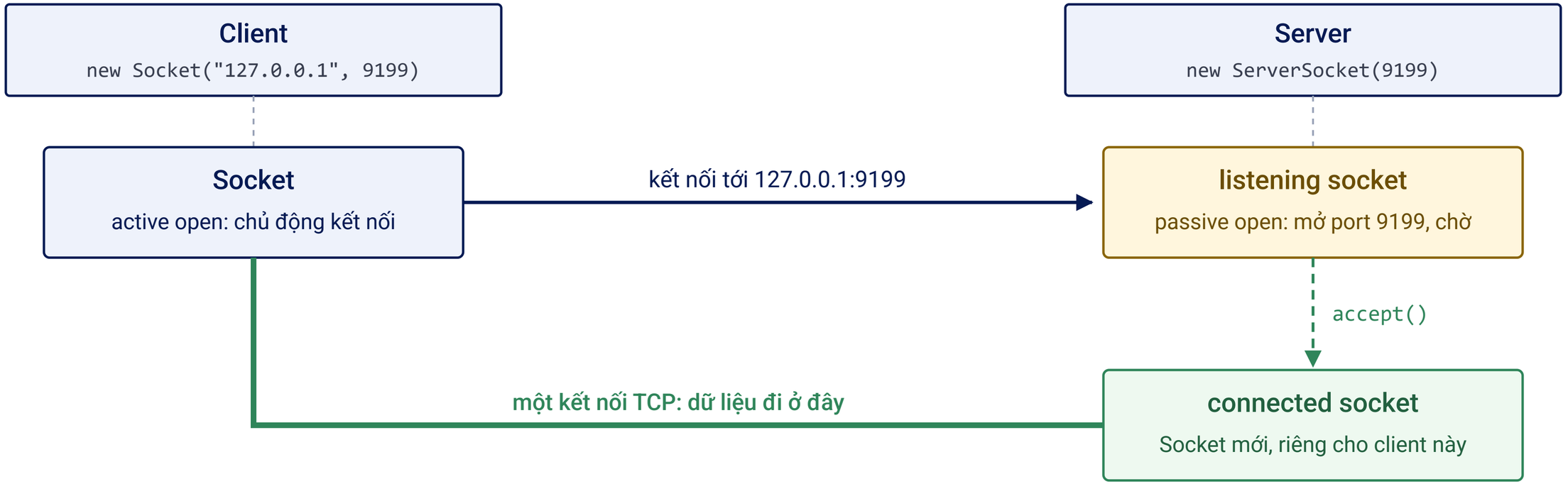
- Thread và race condition
- Synchronized, happens-before và wait/notify
- Executor, thread pool và virtual thread
- Bài toán ứng dụng: server tra cứu phục vụ nhiều client

Đến giờ, NetCat vẫn kết nối hộ chúng ta

1. Phần công cụ kiểm thử: thử giao thức một dòng bằng **NetCat** — công cụ lo phần kết nối
2. Từ phần này: hai đầu là hai chương trình **Java của mình**, phải tự gặp nhau

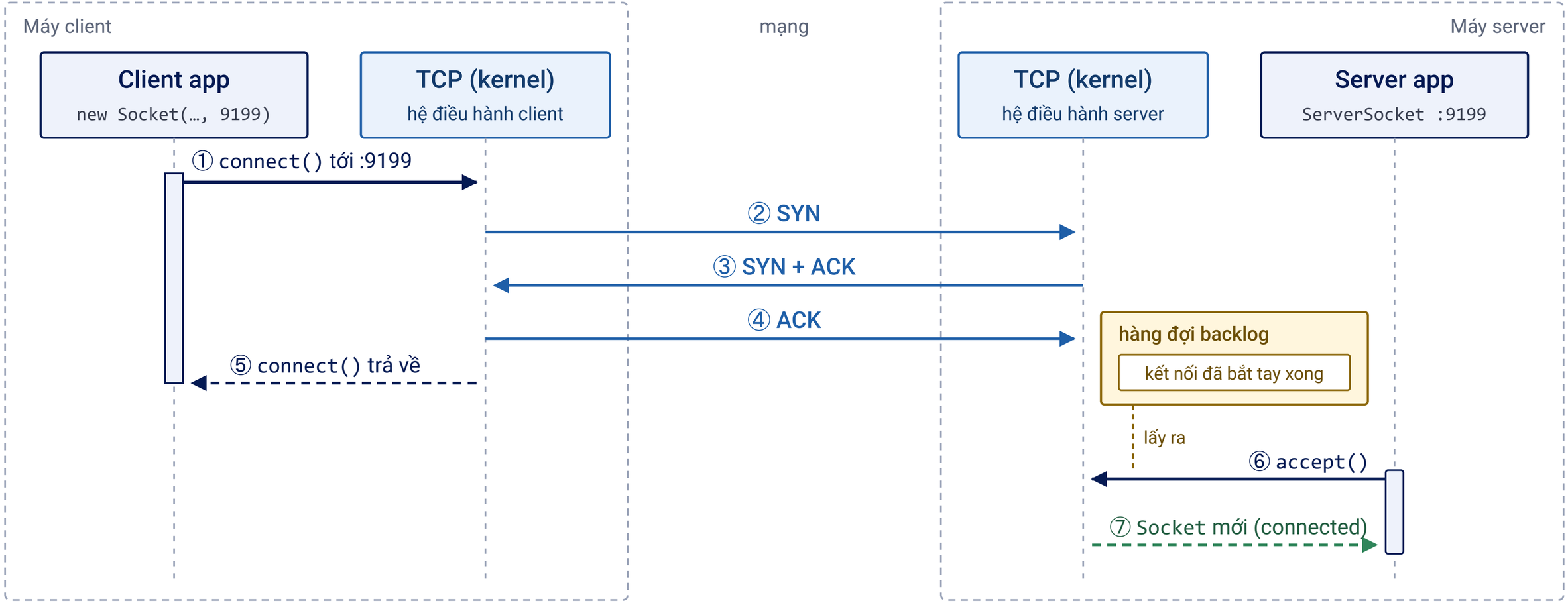
> **Câu hỏi của phần này:** hai chương trình gặp nhau qua TCP như thế nào, và gặp rồi thì mỗi bên cầm gì trong tay?

Server chờ sẵn, client chủ động tìm tới



Server dùng **hai loại** socket: một listening socket để chờ, và mỗi kết nối một connected socket để trao đổi dữ liệu.

Bắt tay do hệ điều hành làm, accept chỉ lấy kết nối ra



`connect()` trả về = bắt tay đã xong. **Chưa chứng minh** server đã gọi `accept()`.

`accept()` không bắt tay. Nó lấy một kết nối đã xong khỏi hàng đợi và trả về connected socket.

Câu hỏi 1

Ở phía client, `connect()` vừa trả về thành công. Điều nào chắc chắn đã xảy ra?

A. Server đã gọi `accept()` và nhận được connected socket.

B. Bắt tay TCP đã hoàn tất; kết nối có thể vẫn đang nằm trong hàng đợi của server.

C. Server đã đọc được dòng đầu tiên client gửi.

D. Server đã cấp một port mới riêng cho kết nối này.

Câu hỏi 1

Ở phía client, `connect()` vừa trả về thành công. Điều nào chắc chắn đã xảy ra?

A. Server đã gọi `accept()` và nhận được connected socket.

B. Bắt tay TCP đã hoàn tất; kết nối có thể vẫn đang nằm trong hàng đợi của server.

C. Server đã đọc được dòng đầu tiên client gửi.

D. Server đã cấp một port mới riêng cho kết nối này.

Bắt tay do TCP trong hệ điều hành hai máy làm. `connect()` trả về khi bắt tay xong; `accept()` chỉ lấy kết nối đã xong ra khỏi hàng đợi và có thể đến sau. Kết nối dùng chung port 9199 của server, không có port mới.

Server: mở port, chờ, đọc một dòng, trả một dòng

Java

```
try (ServerSocket listener = new ServerSocket(9199)) {  // passive open: mở port, chờ
    System.out.println("listening on port " + listener.getLocalPort());
    try (Socket connected = listener.accept()) {  // lấy một kết nối khỏi hàng đợi
        System.out.println("accepted: local port " + connected.getLocalPort()
                + ", client port " + connected.getPort());
        var in = new BufferedReader(new InputStreamReader(connected.getInputStream(), UTF_8));
        var out = new BufferedWriter(new OutputStreamWriter(connected.getOutputStream(), UTF_8));
        String line = in.readLine();  // đọc một dòng
        out.write("echo: " + line + "\n");  // trả lại một dòng
        out.flush();
    }
}
```

Kết quả chạy

```
listening on port 9199
accepted: local port 9199, client port 49419
```

Client: một câu lệnh new Socket là đủ để kết nối

Java

```
try (Socket socket = new Socket("127.0.0.1", 9199)) {  // active open: kết nối tới server
    System.out.println("connected: local port " + socket.getLocalPort()
            + ", server port " + socket.getPort());
    var out = new BufferedWriter(new OutputStreamWriter(socket.getOutputStream(), UTF_8));
    var in = new BufferedReader(new InputStreamReader(socket.getInputStream(), UTF_8));
    out.write("hello\n");  // gửi một dòng
    out.flush();
    System.out.println("reply: " + in.readLine());  // đọc dòng trả về
}
```

Kết quả chạy

```
connected: local port 49419, server port 9199
reply: echo: hello
```

- Cùng một kết nối, nhìn từ hai đầu: local port và remote port **đổi chỗ**

Chạy client khi chưa có server: connection refused

Java

```
try (Socket socket = new Socket("127.0.0.1", 9199)) {  // active open: kết nối tới server
```

Kết quả chạy

```
Exception in thread "main" java.net.ConnectException: Connection refused: connect
        at java.base/sun.nio.ch.Net.connect0(Native Method)
        at java.base/sun.nio.ch.Net.connect(Net.java:601)
        at java.base/sun.nio.ch.Net.connect(Net.java:590)
        at java.base/sun.nio.ch.NioSocketImpl.connect(NioSocketImpl.java:583)
        at java.base/java.net.SocksSocketImpl.connect(SocksSocketImpl.java:327)
        at java.base/java.net.Socket.connect(Socket.java:751)
        at java.base/java.net.Socket.connect(Socket.java:686)
        at java.base/java.net.Socket.<init>(Socket.java:555)
        at java.base/java.net.Socket.<init>(Socket.java:324)
        at netlab.first.FirstClient.main(FirstClient.java:21)
```

- Dòng đầu: **loại lỗi** — máy kia trả lời, nhưng không có ai nghe ở port 9199
- Dòng cuối: dòng đầu tiên thuộc **code của mình** — chỗ gọi gây lỗi

Câu hỏi 2

Bạn chạy FirstClient khi FirstServer chưa chạy, trên cùng một máy. Điều gì xảy ra?

A. Client chờ đến khi server chạy rồi tự kết nối.

B. Client kết nối được, nhưng `readLine()` trả về null.

C. Client nhận ConnectException (connection refused) gần như ngay lập tức.

D. Client tự chuyển sang một port khác còn trống trên máy đó.

Câu hỏi 2

Bạn chạy FirstClient khi FirstServer chưa chạy, trên cùng một máy. Điều gì xảy ra?

A. Client chờ đến khi server chạy rồi tự kết nối.

B. Client kết nối được, nhưng `readLine()` trả về null.

C. Client nhận ConnectException (connection refused) gần như ngay lập tức.

D. Client tự chuyển sang một port khác còn trống trên máy đó.

Không có chương trình nào nghe ở port 9199 nên hệ điều hành từ chối bắt tay ngay và `connect()` ném ConnectException. TCP không tự chờ server xuất hiện, và client không tự đổi port đích.

Tổng kết

1. Server **chờ** bằng listening socket (passive open); client **chủ động** kết nối tới (active open)

2. Bắt tay do hệ điều hành làm; `accept()` lấy kết nối khỏi hàng đợi, trả về **connected socket**

3. Không có ai nghe ở port đích → **connection refused** ngay

Từ khóa cần nhớ socket · ServerSocket · active open · passive open · listening socket · connected socket · bắt tay ba bước · backlog · RST · connection refused

Tiếp theo

TCP: timeout, half-close và lỗi kết nối